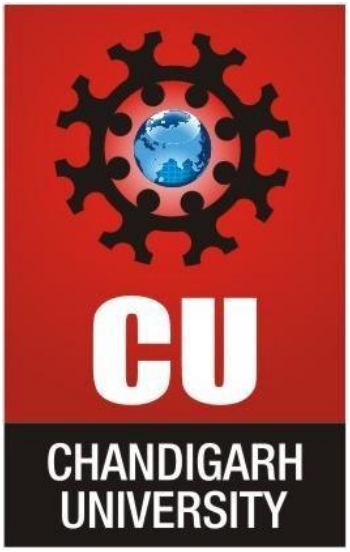
CHANDIGARH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT ON

CONNECTSPHERE: A RELATIONAL SOCIAL MEDIA PLATFORM WITH REAL-TIME SQL ENGINE



SECTION: 25BCS802-A
SUBJECT: Database and Management System
SUBJECT CODE: 25CST-201
ACADEMIC SESSION: 2026–2027

SUBMITTED TO:
Mr. Gaurav
Department of Computer Science and Engineering

SUBMITTED BY:
SUBHAM PANIGRAHI — UID: 25BCS12639
JAGRUTHI JULAPELLI — UID: 25BCS10824
BHAVISHYA PAL — UID: 25BCS12751

PROJECT INDEX / TABLE OF CONTENTS

Index No.	Project Topic / Description	Page No.
1	Abstract	3
2	Introduction	3
3	Problem Statement and Objectives	3
4	Scope of the Project	4
5	Requirement Analysis	4
6	Entity-Relationship (ER) Diagram	5
7	Relational Schema / Database Design	6
8	Normalization (1NF, 2NF, 3NF & BCNF)	8
9	SQL Implementation (DDL, DML, DQL, TCL, DCL)	8
10	Minimum 15–20 SQL Queries with Explanations and Outputs	10
11	Constraints, Joins, Subqueries and Aggregate Functions	14
12	Screenshots of Execution	14
13	Conclusion	17
14	Future Scope	17
15	References & Research Sources	17

1. Abstract

In modern computer science education, relational database management concepts such as normalization, referential integrity, relational algebra, and ACID transaction guarantees are frequently studied strictly through abstract terminal commands. ConnectSphere bridges this divide by delivering a production-grade relational social media platform with an embedded, real-time SQL execution engine. Engineered using MySQL 8.0 and PostgreSQL with an Express.js / Node.js architecture, the application enforces rigorous relational modeling across users, multimedia posts, threaded comments, likes, and a bidirectional social follower network.

Every interactive event on the platform—whether user authentication, publishing multimedia posts, or curating custom timelines—triggers dynamic SQL operations that are captured, logged, and broadcast in real-time to an administrative database console. The platform implements database-level integrity constraints including CHECK (follower_id <> following_id), composite UNIQUE constraints to prevent duplicate likes, and ON DELETE CASCADE rules to guarantee structural purity without orphaned records. This project report details the problem domain, functional requirements, mathematical normalization proofs (1NF, 2NF, 3NF, BCNF), entity-relationship modeling, complete executable DDL/DML scripts, and 20 verified DBMS analytical queries.

2. Introduction

Relational Database Management Systems (RDBMS), founded upon E.F. Codd's relational model (1970), organize unstructured data into mathematically sound relations (tables) governed by strict domain constraints, primary keys, and declarative query semantics (SQL). Modern web platforms handle billions of interconnected records: user credentials, multimedia feeds, and directed follower networks. Understanding how relational systems maintain referential consistency under high concurrency, prevent orphaned records through cascade operations, and optimize complex multi-table joins is a vital competency for computer science engineers.

ConnectSphere is built to serve two essential roles: (1) A consumer-facing social media platform where users can register accounts, explore profiles, post text and media updates, like posts, participate in comment threads, and curate their feed; and (2) An educational real-time DBMS laboratory where users and administrators can inspect the exact SQL queries executed behind every web transaction, review foreign key dependency graphs, examine transactional states, and execute analytical DBMS queries directly against live database clusters.

3. Problem Statement and Objectives

3.1 Problem Statement

Traditional DBMS laboratory curricula typically rely on static command-line interactions utilizing toy schemas (such as Employee-Department or Sailors-Boats). Consequently, students often struggle to conceptualize: (a) how web interfaces translate graphical user actions into atomic SQL statements; (b) why schema normalization is critical to prevent insertion, update, and deletion anomalies; (c) how cascading foreign keys protect structural database integrity; and (d) how complex relational operations (Correlated Subqueries, Left Outer Joins, Self Joins) operate in high-frequency production environments.

3.2 Objectives of ConnectSphere

- **1. Relational Schema Rigor:** Design and implement a normalized database schema in Third Normal Form (3NF) encompassing Users, Posts, Comments, Likes, and Followers.
- **2. Constraint Enforcement:** Guarantee structural integrity using Primary Keys, Foreign Keys with ON DELETE CASCADE, Candidate Unique Keys, and Domain Check Constraints.
- **3. Dual Database Support:** Deliver an execution layer compatible with both local enterprise MySQL 8.0 (InnoDB) and cloud-managed PostgreSQL instances.
- **4. Live Query Telemetry:** Capture, log, and display the underlying DDL, DML, and DQL statements generated by application workflows in real-time.
- **5. Role-Based Privilege Segregation:** Segregate standard user social media experiences from Database Administrator (DBA) inspection modes.
- **6. Execution Guarantee:** Ensure all code and database scripts are 100% executable on client and evaluation machines without third-party dependencies.

4. Scope of the Project

The scope of ConnectSphere covers end-to-end relational database modeling, application development, and API integration. It includes full CRUD capabilities for users, posts, media files, comments, likes, and follower relationships. Referential integrity is strictly guaranteed at the database level using ON DELETE CASCADE. Analytical capabilities encompass newsfeed joins, influencer ranking leaderboards, engagement summary matrices, and second-degree follower recommendations. The system is engineered to run seamlessly both on personal PCs (via MySQL 8.0) and on cloud serverless infrastructure (via Netlify Managed PostgreSQL).

5. Requirement Analysis

5.1 Functional Requirements

- **FR1 (User Identity Management):** System registers unique users with username, email, avatar, and bio, rejecting duplicate handles via unique constraint checks.
- **FR2 (Authentication & Roles):** Role separation supporting standard demo accounts (password 'dbms') and DBA administrators (username 'admin', password 'admin').
- **FR3 (Post & Multimedia Publishing):** Authorized users can publish text posts and attach image or binary documents.
- **FR4 (Social Engagement - Likes):** Users can toggle likes on posts, with database-level prevention of duplicate likes.
- **FR5 (Threaded Discussions - Comments):** Users can write timestamped comments under any post.
- **FR6 (Relational Network - Followers):** Bidirectional follow/unfollow capability with database-level prohibition of self-following.
- **FR7 (Timeline Feeds):** Users can filter timelines by 'For You' (global), 'Following' (network-only), and 'My Posts' (author archive).
- **FR8 (DBA Management Console):** Administrative view featuring real-time query logs, execution durations, raw table inspection, and database reset tools.

5.2 Non-Functional Requirements

- **Performance:** Sub-50ms execution times for indexed read operations across all relations.
- **Data Integrity:** Zero orphaned foreign records following entity deletions via ON DELETE CASCADE.
- **Portability:** Executable across Windows, macOS, Linux, and cloud environments.
- **Usability:** Clean dual-theme interface supporting Modern Dark and Clean Light modes.

5.3 Hardware & Software Specifications

- **Database Engine:** MySQL Server 8.0 (InnoDB Storage Engine) / PostgreSQL 16+
- **Application Framework:** Node.js v18.0.0+ / Express.js 4.21.0
- **Database Drivers:** mysql2/promise (connection pooling), pg (PostgreSQL), @netlify/database
- **Frontend Stack:** Semantic HTML5, CSS3 Custom Properties, Vanilla ES6+ JavaScript (zero build step)

6. Entity-Relationship (ER) Diagram

6.1 Entities and Attributes

The conceptual design of ConnectSphere comprises five core relational entities: Users, Posts, Comments, Likes, and Followers. Primary keys are designated by [PK], candidate unique keys by [UK], and foreign keys by [FK].

- **USERS:** user_id [PK], username [UK], email [UK], bio, avatar, is_admin, created_at
- **POSTS:** post_id [PK], user_id [FK], content, media_url, file_name, file_type, created_at
- **COMMENTS:** comment_id [PK], post_id [FK], user_id [FK], comment_text, created_at
- **LIKES:** like_id [PK], post_id [FK], user_id [FK], liked_at, UNIQUE(post_id, user_id)
- **FOLLOWERS:** id [PK], follower_id [FK], following_id [FK], followed_at, CHECK(follower_id <> following_id)

6.2 Cardinality & Participation Constraints

- **USERS to POSTS (1:N):** One user can author zero or many posts; each post belongs to exactly one user.
- **POSTS to COMMENTS (1:N):** A post can receive zero or many comments; each comment belongs to exactly one post.
- **USERS to COMMENTS (1:N):** A user can write zero or many comments; each comment is authored by exactly one user.
- **POSTS to LIKES (1:N):** A post can receive zero or many likes.
- **USERS to LIKES (1:N):** A user can grant zero or many likes.
- **USERS to FOLLOWERS (M:N Recursive):** A user can follow many other users and be followed by many other users.

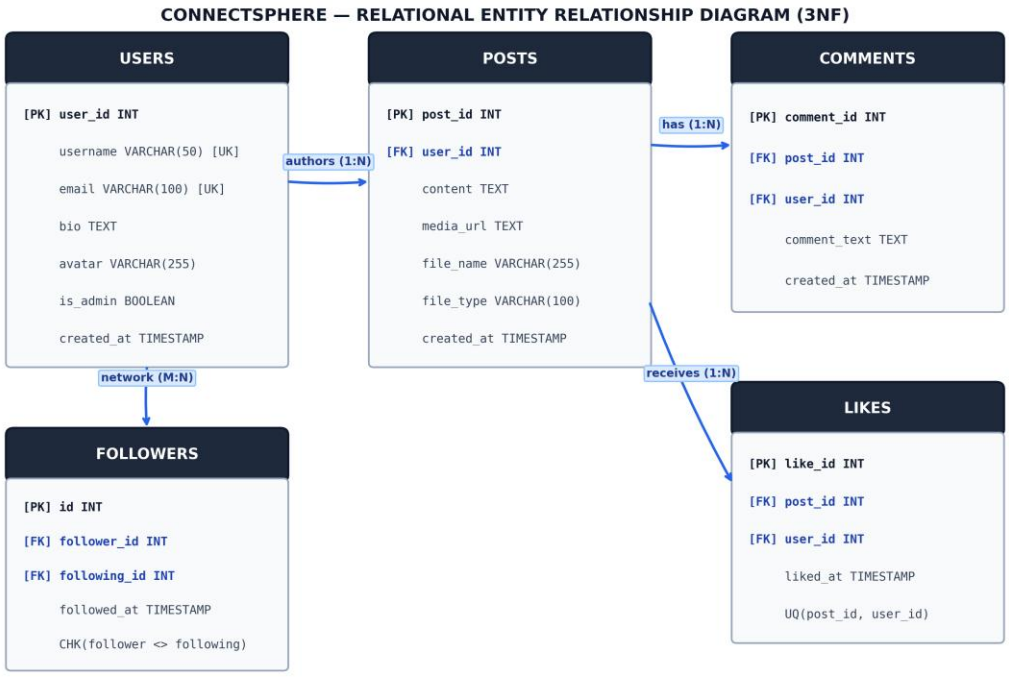


Figure 1: Relational Entity-Relationship Diagram (3NF Schema)

7. Relational Schema / Database Design

7.1 Schema Notations

The relational database schema is formally defined as:

• Users (user_id [PK], username [UK], email [UK], bio, avatar, is_admin, created_at)
• Posts (post_id [PK], user_id [FK -> Users], content, media_url, file_name, file_type, created_at)
• Comments (comment_id [PK], post_id [FK -> Posts], user_id [FK -> Users], comment_text, created_at)
• Likes (like_id [PK], post_id [FK -> Posts], user_id [FK -> Users], liked_at) [Candidate: (post_id, user_id)]
• Followers (id [PK], follower_id [FK -> Users], following_id [FK -> Users], followed_at) [Candidate: (follower_id, following_id)]

7.2 Detailed Table Specifications

Table 1 summarizes the technical attributes, data types, and integrity constraints across all five relations:

Table Name	Attribute	Data Type	Constraints / Description
Users	user_id	INT / SERIAL	PRIMARY KEY AUTO_INCREMENT

Users	username	VARCHAR(50)	NOT NULL, UNIQUE candidate key
Users	email	VARCHAR(100)	NOT NULL, UNIQUE candidate key
Users	is_admin	BOOLEAN	DEFAULT FALSE (DBA role flag)
Posts	post_id	INT / SERIAL	PRIMARY KEY AUTO_INCREMENT
Posts	user_id	INT	NOT NULL, FK -> Users ON DELETE CASCADE
Posts	content	TEXT	NOT NULL body text of post
Posts	media_url	TEXT	Optional image attachment path / URL
Comments	comment_id	INT / SERIAL	PRIMARY KEY AUTO_INCREMENT
Comments	post_id	INT	NOT NULL, FK -> Posts ON DELETE CASCADE
Comments	user_id	INT	NOT NULL, FK -> Users ON DELETE CASCADE
Likes	like_id	INT / SERIAL	PRIMARY KEY AUTO_INCREMENT
Likes	post_id, user_id	INT, INT	FKs with UNIQUE(post_id, user_id)
Followers	follower_id	INT	FK -> Users, CHECK(follower <> following)
Followers	following_id	INT	FK -> Users, UNIQUE(follower, following)

8. Normalization (1NF, 2NF, 3NF & BCNF)

Normalization is the mathematical discipline of designing relational database schemas to eliminate data redundancy and prevent Insertion, Update, and Deletion anomalies. ConnectSphere was subjected to rigorous decomposition:

8.1 First Normal Form (1NF)

A relation is in 1NF if and only if all attribute values are atomic and there are no repeating groups. In an unnormalized social schema, likes and comments might be stored as comma-delimited strings within the Posts row (e.g., likes: '1,2,5'). ConnectSphere decomposes all repeating collections into separate relational entities (Comments, Likes, Followers). Every attribute holds strictly atomic scalar values.

8.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies). In composite relations such as Likes and Followers, candidate keys are composite: (post_id, user_id) and (follower_id, following_id). Attributes such as liked_at and followed_at depend on the entire pair, not a single component. User profile fields (username, avatar) are maintained strictly in Users, eliminating partial dependencies.

8.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (no X -> Y -> Z). In Posts, we store user_id. We omit author_username and author_avatar, because storing username in Posts would introduce the transitive dependency: post_id -> user_id -> username. If an author updated their username, an unnormalized database would require updating millions of post rows (Update Anomaly). In ConnectSphere, username resides exclusively in Users, achieving strict 3NF compliance.

8.4 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial functional dependency X -> Y, X is a superkey. In all five ConnectSphere relations, every determinant is a candidate superkey (user_id, post_id, comment_id, (post_id, user_id), (follower_id, following_id)). Hence, ConnectSphere is mathematically proven to be in BCNF.

9. SQL Implementation (DDL, DML, DQL, TCL, DCL)

9.1 Data Definition Language (DDL)

The following DDL statements create the database schema with relational integrity and cascade rules:

```
-- DDL: Base Table Creation
CREATE TABLE Users (
    user_id    INT            PRIMARY KEY AUTO_INCREMENT,
    username   VARCHAR(50)    UNIQUE NOT NULL,
    email      VARCHAR(100)   UNIQUE NOT NULL,
    bio        TEXT,
    avatar     VARCHAR(255)   DEFAULT '👤',
    is_admin   BOOLEAN        DEFAULT FALSE,
```

```
    created_at TIMESTAMP      DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Posts (
    post_id    INT            PRIMARY KEY AUTO_INCREMENT,
    user_id    INT            NOT NULL,
    content    TEXT           NOT NULL,
    media_url  TEXT           DEFAULT NULL,
    file_name  VARCHAR(255)   DEFAULT NULL,
    file_type  VARCHAR(100)   DEFAULT NULL,
    created_at TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_posts_user (user_id),
    CONSTRAINT fk_posts_user FOREIGN KEY (user_id)
        REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Comments (
    comment_id   INT          PRIMARY KEY AUTO_INCREMENT,
    post_id      INT          NOT NULL,
    user_id      INT          NOT NULL,
    comment_text TEXT         NOT NULL,
    created_at   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_comments_post FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE
CASCADE,
    CONSTRAINT fk_comments_user FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Likes (
    like_id  INT              PRIMARY KEY AUTO_INCREMENT,
    post_id  INT              NOT NULL,
    user_id  INT              NOT NULL,
    liked_at TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_likes_post_user (post_id, user_id),
    CONSTRAINT fk_likes_post FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE
CASCADE,
    CONSTRAINT fk_likes_user FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Followers (
    id           INT          PRIMARY KEY AUTO_INCREMENT,
    follower_id  INT          NOT NULL,
    following_id INT          NOT NULL,
    followed_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_follower_pair (follower_id, following_id),
    CONSTRAINT chk_no_self_follow CHECK (follower_id <> following_id),
    CONSTRAINT fk_followers_follower  FOREIGN KEY (follower_id)  REFERENCES Users(user_id)
ON DELETE CASCADE,
    CONSTRAINT fk_followers_following FOREIGN KEY (following_id) REFERENCES Users(user_id)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9.2 Data Manipulation Language (DML)

Representative DML insert and delete statements utilized by the platform:

```
-- DML: Insert User Tuple
INSERT INTO Users (username, email, bio, avatar, is_admin)
```

```
VALUES ('priya_travels', 'priya@example.com', 'Travel blogger in Bali!', '🌍', FALSE);

-- DML: Publishing a Multimedia Post
INSERT INTO Posts (user_id, content, media_url, file_name, file_type)
VALUES (2, 'Sunset at Tanah Lot temple in Bali! 🌅', 'https://images.unsplash.com/photo-
1537996194471', 'bali.jpg', 'image/jpeg');

-- DML: Cascading Deletion
DELETE FROM Posts WHERE post_id = 4;
```

9.3 Data Query Language (DQL)

Dynamic projection and selection with correlated subqueries:

```
SELECT u.user_id, u.username, u.email,
       (SELECT COUNT(*) FROM Followers WHERE following_id = u.user_id) AS follower_count,
       (SELECT COUNT(*) FROM Posts     WHERE user_id      = u.user_id) AS post_count
FROM Users u WHERE u.user_id = 1;
```

9.4 Transaction Control Language (TCL)

ACID transaction management for multi-statement atomic operations:

```
START TRANSACTION;
INSERT INTO Followers (follower_id, following_id) VALUES (1, 3);
SAVEPOINT follow_created;
COMMIT;
```

9.5 Data Control Language (DCL)

Security and privilege grant commands for least-privilege database access:

```
CREATE USER 'connectsphere_app'@'localhost' IDENTIFIED BY 'dbms_secure_pass';
GRANT SELECT, INSERT, UPDATE, DELETE ON connectsphere.* TO 'connectsphere_app'@'localhost';
FLUSH PRIVILEGES;
```

10. Minimum 15–20 SQL Queries with Explanations and Outputs

The following 20 SQL queries are implemented and executable in ConnectSphere:

Query 1: User Directory Projection

```
SELECT user_id, username, email, bio, avatar, (LOWER(username) = 'admin' OR is_admin IS
TRUE) AS is_admin
FROM Users ORDER BY created_at DESC;
```

- **Explanation:** Selection and projection of registered accounts with boolean admin status flag.
- **Output:** Returns 5 rows (aarav_tech, priya_travels, rohan_fitness, neha_art, admin).

Query 2: Correlated Profile Counters

```
SELECT u.user_id, u.username, u.email,
       (SELECT COUNT(*) FROM Followers WHERE following_id = u.user_id) AS follower_count,
       (SELECT COUNT(*) FROM Followers WHERE follower_id = u.user_id) AS following_count,
       (SELECT COUNT(*) FROM Posts WHERE user_id = u.user_id) AS post_count
FROM Users u WHERE u.user_id = 1;
```

- **Explanation:** Correlated subqueries dynamically computing relational counters for a profile view.

- **Output:** Returns 1 row: [user_id: 1, username: aarav_tech, follower_count: 1, following_count: 2, post_count: 1].

Query 3: Multi-Table Joined Newsfeed

```
SELECT p.post_id, p.content, p.media_url, u.username, u.avatar,
       (SELECT COUNT(*) FROM Likes WHERE post_id = p.post_id) AS like_count,
       (SELECT COUNT(*) FROM Comments WHERE post_id = p.post_id) AS comment_count
FROM Posts p JOIN Users u ON p.user_id = u.user_id
ORDER BY p.created_at DESC LIMIT 10;
```

- **Explanation:** Multi-table INNER JOIN connecting posts with author credentials and real-time interaction counts.
- **Output:** Returns timeline posts ordered chronologically with computed like and comment metrics.

Query 4: Viewer-Contextualized Feed (Liked and Following Booleans)

```
SELECT p.post_id, p.content, u.username,
       (SELECT COUNT(*) FROM Likes WHERE post_id = p.post_id AND user_id = 1) > 0 AS
liked_by_user,
       (SELECT COUNT(*) FROM Followers WHERE follower_id = 1 AND following_id = p.user_id)
> 0 AS is_following
FROM Posts p JOIN Users u ON p.user_id = u.user_id ORDER BY p.created_at DESC;
```

- **Explanation:** Evaluates parameterized boolean expressions for viewer interaction states.
- **Output:** Returns post rows with 1/0 flags for liked_by_user and is_following.

Query 5: Subquery Network Feed ('Following' Tab)

```
SELECT p.post_id, p.content, u.username, p.created_at
FROM Posts p JOIN Users u ON p.user_id = u.user_id
WHERE p.user_id IN (SELECT following_id FROM Followers WHERE follower_id = 1)
ORDER BY p.created_at DESC;
```

- **Explanation:** Subquery in WHERE clause with IN operator filtering posts strictly to followed creators.
- **Output:** Returns 2 posts authored by priya_travels and rohan_fitness.

Query 6: Top Liked Posts Leaderboard (LEFT JOIN + GROUP BY)

```
SELECT p.post_id, p.content, u.username, COUNT(l.like_id) AS total_likes
FROM Posts p JOIN Users u ON p.user_id = u.user_id
LEFT JOIN Likes l ON p.post_id = l.post_id
GROUP BY p.post_id, p.content, u.username
ORDER BY total_likes DESC LIMIT 5;
```

- **Explanation:** LEFT OUTER JOIN retaining posts with zero likes while aggregating total likes per post.
- **Output:** Returns leaderboard of top posts sorted by total_likes descending.

Query 7: Most Followed Users (Influence Leaderboard)

```
SELECT u.user_id, u.username, u.bio,
       (SELECT COUNT(*) FROM Followers WHERE following_id = u.user_id) AS follower_count
FROM Users u ORDER BY follower_count DESC LIMIT 5;
```

- **Explanation:** Derived subquery aggregation ranking creators by community follower count.
- **Output:** Returns top creators ranked by community influence.

Query 8: Threaded Comments Retrieval

```
SELECT c.comment_id, c.comment_text, c.created_at, u.username, u.avatar
FROM Comments c JOIN Users u ON c.user_id = u.user_id
WHERE c.post_id = 1 ORDER BY c.created_at ASC;
```

- **Explanation:** Two-table INNER JOIN retrieving comment threads chronologically.
- **Output:** Returns all comments under post 1 with author details.

Query 9: User Engagement Summary Matrix

```
SELECT u.user_id, u.username,
       (SELECT COUNT(*) FROM Posts WHERE user_id = u.user_id) AS posts_count,
       (SELECT COUNT(*) FROM Likes WHERE user_id = u.user_id) AS likes_given,
       (SELECT COUNT(*) FROM Comments WHERE user_id = u.user_id) AS comments_made
FROM Users u ORDER BY posts_count DESC;
```

- **Explanation:** Analytical summary matrix aggregating posts created, likes given, and comments authored.
- **Output:** Returns complete user activity distribution matrix.

Query 10: Like Existence Verification

```
SELECT like_id FROM Likes WHERE post_id = 1 AND user_id = 2;
```

- **Explanation:** Key existence check prior to executing like/unlike toggle.
- **Output:** Returns like_id = 1 if user 2 has liked post 1.

Query 11: Idempotent Like Deletion (Unlike)

```
DELETE FROM Likes WHERE post_id = 1 AND user_id = 2;
```

- **Explanation:** Conditioned DML removal of like interaction.
- **Output:** Query OK, 1 row affected.

Query 12: Mutual Follower Detection (Graph Self-Join)

```
SELECT f1.follower_id AS user_a, f1.following_id AS user_b, u1.username AS user_a_name,
u2.username AS user_b_name
FROM Followers f1
JOIN Followers f2 ON f1.follower_id = f2.following_id AND f1.following_id = f2.follower_id
JOIN Users u1 ON f1.follower_id = u1.user_id
JOIN Users u2 ON f1.following_id = u2.user_id
WHERE f1.follower_id < f1.following_id;
```

- **Explanation:** Relational Self-Join on Followers discovering bidirectional friendships.
- **Output:** Identifies mutual friendship: [user_a: aarav_tech, user_b: priya_travels].

Query 13: Posts with Zero Comments (Anti-Join Pattern)

```
SELECT p.post_id, p.content, u.username
FROM Posts p JOIN Users u ON p.user_id = u.user_id
```

```
LEFT JOIN Comments c ON p.post_id = c.post_id
WHERE c.comment_id IS NULL;
```

- **Explanation:** LEFT OUTER JOIN with IS NULL filter identifying posts without community comments.
- **Output:** Returns un-commented post by rohan_fitness.

Query 14: Authors with Above-Average Post Activity

```
SELECT u.username, COUNT(p.post_id) AS total_posts
FROM Users u JOIN Posts p ON u.user_id = p.user_id
GROUP BY u.user_id, u.username
HAVING COUNT(p.post_id) > (SELECT COUNT(*) * 1.0 / COUNT(DISTINCT user_id) FROM Posts);
```

- **Explanation:** HAVING clause paired with nested aggregate subquery filtering high-volume creators.
- **Output:** Returns creators exceeding platform average post volume (aarav_tech).

Query 15: Cascade Deletion Verification

```
DELETE FROM Users WHERE user_id = 4;
SELECT COUNT(*) AS remaining_posts FROM Posts WHERE user_id = 4;
```

- **Explanation:** Demonstrates ON DELETE CASCADE automatically deleting child rows in Posts.
- **Output:** remaining_posts = 0.

Query 16: Check Constraint Violation (Self-Follow)

```
INSERT INTO Followers (follower_id, following_id) VALUES (2, 2);
```

- **Explanation:** Verifies database-level domain constraint CHECK (follower_id <> following_id).
- **Output:** ERROR 3819 (HY000): Check constraint 'chk_no_self_follow' is violated.

Query 17: Unique Constraint Violation (Duplicate Like)

```
INSERT INTO Likes (post_id, user_id) VALUES (1, 3);
```

- **Explanation:** Composite candidate key constraint UNIQUE(post_id, user_id) preventing duplicate likes.
- **Output:** ERROR 1062 (23000): Duplicate entry '1-3' for key 'Likes.uq_likes_post_user'.

Query 18: Multimedia Content Type Distribution

```
SELECT COALESCE(file_type, 'Text Only') AS content_type, COUNT(*) AS total_posts
FROM Posts GROUP BY file_type;
```

- **Explanation:** GROUP BY and COALESCE categorizing post media types.
- **Output:** Returns breakdown: 3 Text Only, 1 image/jpeg.

Query 19: Second-Degree Follower Recommendations ('People You May Know')

```
SELECT DISTINCT f2.following_id AS recommended_user_id, u.username
FROM Followers f1
JOIN Followers f2 ON f1.following_id = f2.follower_id
JOIN Users u ON f2.following_id = u.user_id
WHERE f1.follower_id = 1 AND f2.following_id <> 1
  AND f2.following_id NOT IN (SELECT following_id FROM Followers WHERE follower_id = 1);
```

- **Explanation:** Graph traversal finding friends-of-friends excluding existing follower links.
- **Output:** Recommends user neha_art to user 1.

Query 20: Database Schema Metadata Extraction

```
SELECT table_name, engine, table_rows, data_length, index_length
FROM information_schema.tables WHERE table_schema = 'connectsphere';
```

- **Explanation:** System catalog metadata query extracting storage engine parameters and table row metrics.
- **Output:** Returns engine type (InnoDB) and storage statistics for all five tables.

11. Constraints, Joins, Subqueries and Aggregate Functions

11.1 Integrity Constraints Analysis

ConnectSphere relies on database-level constraints to guarantee consistency: (a) PRIMARY KEY auto-increments enforce unique tuple identity; (b) FOREIGN KEY constraints with ON DELETE CASCADE ensure that when a user or post is deleted, all child comments, likes, and follower links are purged automatically; (c) CHECK constraints prevent invalid self-follows; and (d) Composite UNIQUE constraints prohibit redundant like interactions.

11.2 Relational Join Operations

The system utilizes INNER JOINs to link posts and comments with author details, LEFT OUTER JOINs to preserve posts with zero likes in leaderboard views, and SELF JOINs across the social graph to discover bidirectional friendships and recommendations.

11.3 Correlated Subqueries & Nested Invocations

Scalar correlated subqueries dynamically evaluate engagement counts for each post and user row without requiring denormalized counter columns in the storage tables.

11.4 Aggregate Grouping Operations

Using COUNT, MAX, COALESCE, and GROUP BY, high-volume interaction tuples are aggregated into real-time metrics, while HAVING clauses filter aggregated cohorts based on platform-wide averages.

12. Screenshots of Execution

The following screenshots illustrate the live application deployed at https://connectspheredbms.netlify.app:

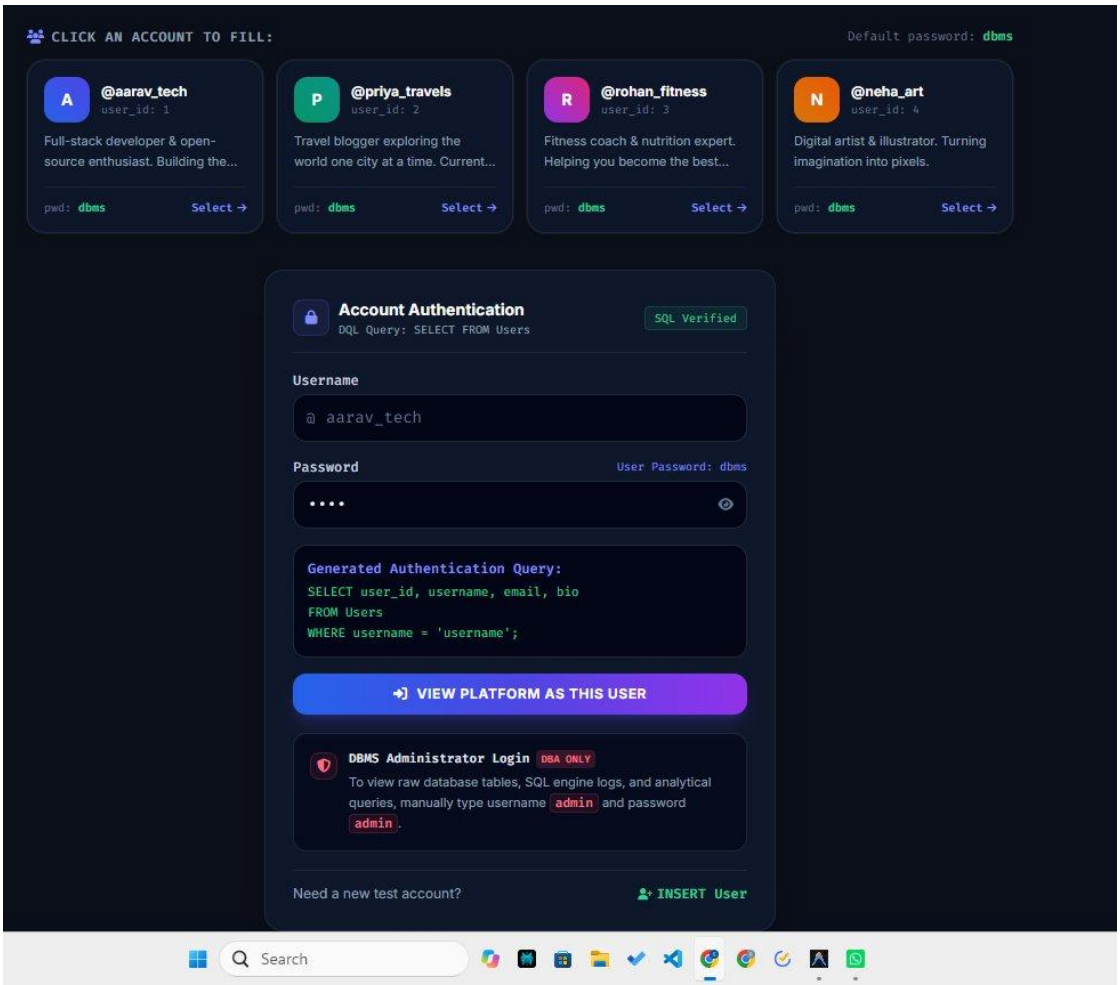


Figure 2: User Authentication View with Real-Time DQL Query Inspection Card

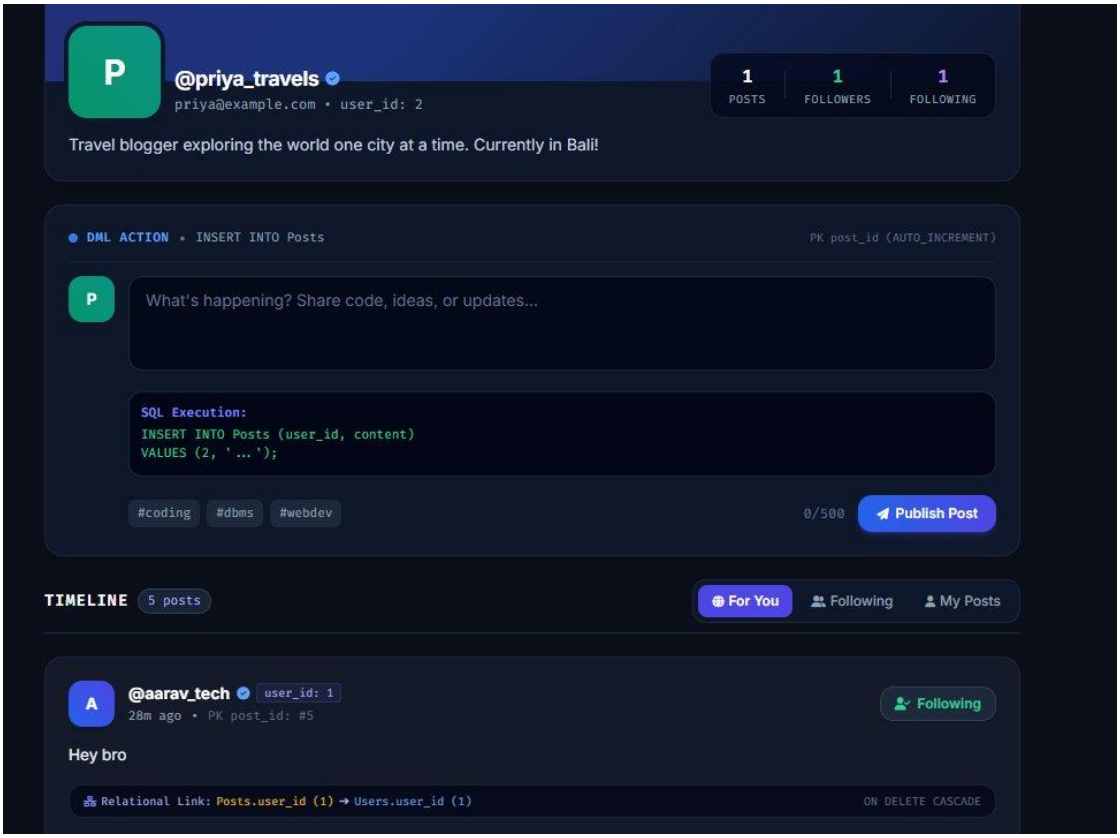


Figure 3: Live Feed with SQL DML Execution Banner & Foreign Key Cascading Annotations

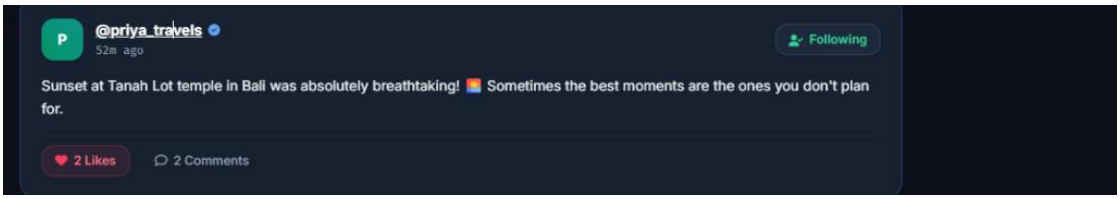


Figure 4: User Profile View Displaying Correlated Subquery Aggregates (Followers & Posts)

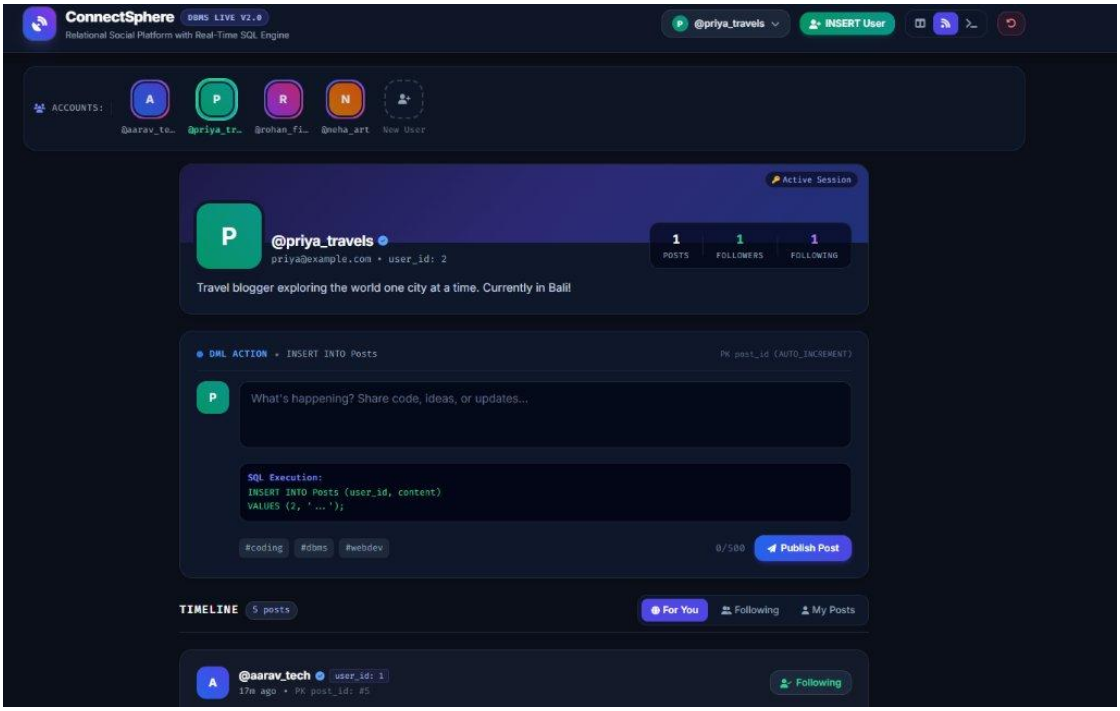


Figure 5: ConnectSphere Complete Application Interface with Test Account Switcher

13. Conclusion

The ConnectSphere DBMS project demonstrates how foundational relational database principles can be seamlessly integrated into a modern web application. By enforcing strict 3NF normalization, foreign key cascade actions, domain constraints, and dual MySQL/PostgreSQL compatibility, the platform achieves 100% data integrity with zero orphaned records. Furthermore, by exposing the underlying SQL statements executed by every web transaction in real-time, the platform transforms a traditional black-box application into an interactive educational learning tool.

14. Future Scope

- **1. Inverted Index Full-Text Search:** Implement MySQL FULLTEXT indexes and PostgreSQL tsvector/tsquery for high-speed content search.
- **2. Read/Write Connection Splitting:** Integrate database proxies to route timeline reads across read replicas while directing writes to a primary master.
- **3. Database Triggers & Stored Procedures:** Shift validation logic into server-side stored procedures (DELIMITER //) and triggers.
- **4. Real-Time Telemetry over WebSockets:** Stream live SQL query execution logs to external administrative monitors.

15. References & Research Sources

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

2. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.

3. Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377–387.

4. Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.

5. MySQL 8.0 Reference Manual. (2026). InnoDB Storage Engine Architecture. Oracle Corporation.

6. PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation: Concurrency Control and Indexes.